P0323R12
std::expected

Published Proposal, 2022-01-07

This version:
https://wg21.link/P0323R11

Authors:
Vicente Botet (Nokia)
JF Bastien (Woven Planet)
Jonathan Wakely (Red Hat)

Audience:
LWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:
github.com/jfbastien/papers/blob/master/source/P0323R11.bs

Table of Contents

Class template `expected<T, E>` is a vocabulary type which contains an expected value of type `T`, or an error `E`. The class skews towards behaving like a `T`, because its intended use is when the expected type is contained. When something unexpected occurs, more typing is required. When all is good, code mostly looks as if a `T` were being handled.

Class template `expected<T, E>` contains either:

- A value of type `T`, the expected value type; or
- A value of type `E`, an error type used when an unexpected outcome occured.

The interface can be queried as to whether the underlying value is the expected value (of type `T`) or an unexpected value (of type `E`). The original idea comes from Andrei Alexandrescu C++ and Beyond 2012: Systematic Error Handling in C++ Alexandrescu.Expected, which he revisited in CppCon 2018, including mentions of this paper.

The interface and the rational are based on `std::optional` [N3793]. We consider `expected<T, E>` as a supplement to `optional<T>`, expressing why an expected value isn't contained in the object.

§ 1. Revision History

This paper updates [P0323r10] following a few LWG reviews on 2021-09-10 and later. Remove conversions from `std::unexpected`. Rationalize constraints and effects throughout. Define separate partial specialization for `expected<cv void, E>`. Many other small edits.

In the 2021-04-06 LEWG telecon, LEWG decided to target the IS instead of a TS.

Poll: [P0323r9] should add a feature test macro, change the namespace/header from `experimental` to `std` to be re-targeted to the IS, forward it to electronic polling to forward to LWG

SF	F	N	A	SA
6	6	5	1	0

Various iterations of this paper have been reviewed by WG21 over time:

- LWG Telecon 2021
- LEWG Telecon 2021
- LWG Jacksonville 2018
- LWG Rapperswil 2018
- EWG Albuquerque 2017
- LEWG Albuquerque 2017
- LEWG Toronto 2017
- LEWG Oulu 2016
- SG14 GDC 2016
- LEWG Rapperswil 2014

Related papers:

- This proposal dates back to [N4015] and [N4109].
- [P0323r3] was the last paper with a rationale. LEWG asked that the rationale be dropped from r4 onwards. The rationale is now back in r10, to follow the new LEWG policy of preserving rationales.
- Some design issues were discussed by LWG and answered in [P1051r0].
- [P0650r2] proposes monadic interfaces for `expected` and other types.
- [P0343r1] proposes meta-programming high-order functions.
- [P0262r1] is a related proposal for status / optional value.
- [P0157R0] describes when to use each of the different error report mechanism.
- [P0762r0] discussed how this paper interacts with boost.Outcome.
- [P1095r0] proposes zero overhead deterministic failure.
- [P0709r4] discussed zero-overhead deterministic exceptions.
- [P1028R3] proposes `status_code` and standard `error` object.

§ 2. Motivation

C++'s two main error mechanisms are exceptions and return codes. Characteristics of a good error mechanism are:

1. **Error visibility:** Failure cases should appear throughout the code review: debugging can be painful if errors are hidden.
2. **Information on errors:** Errors should carry information from their origin, causes and possibly the ways to resolve it.
3. **Clean code:** Treatment of errors should be in a separate layer of code and as invisible as possible. The reader could notice the presence of exceptional cases without needing to stop reading.
4. **Non-Intrusive error:** Errors should not monopolize the communication channel dedicated to normal code flow. They must be as discrete as possible. For instance, the return of a function is a channel that should not be exclusively reserved for errors.

The first and the third characteristic seem contradictory and deserve further explanation. The former points out that errors not handled should appear clearly in the code. The latter tells us that error handling must not interfere with the legibility, meaning that it clearly shows the normal execution flow.

Here is a comparison between the exception and return codes:

	Exception	Return error code
Visibility	Not visible without further analysis of the code. However, if an exception is thrown, we can follow the stack trace.	Visible at the first sight by watching the prototype of the called function. However ignoring return code can lead to undefined results and it can be hard to figure out the problem.
Informations	Exceptions can be arbitrarily rich.	Historically a simple integer. Nowadays, the header <system_error> provides richer error code.
Clean code	Provides clean code, exceptions	Force you to add, at least, a if statement after each

	can be completely invisible for the caller.	function call.
Non-Intrusive	Proper communication channel.	Monopolization of the return channel.

We can do the same analysis for the `expected<T, E>` class and observe the advantages over the classic error reporting systems.

1. **Error visibility:** It takes the best of the exception and error code. It's visible because the return type is `expected<T, E>` and users cannot ignore the error case if they want to retrieve the contained value.
2. **Information:** Arbitrarily rich.
3. **Clean code:** The monadic interface of expected provides a framework delegating the error handling to another layer of code. Note that `expected<T, E>` can also act as a bridge between an exception-oriented code and a nothrow world.
4. **Non-Intrusive:** Use the return channel without monopolizing it.

Other notable characteristics of `expected<T, E>` include:

- Associates errors with computational goals.
- Naturally allows multiple errors inflight.
- Teleportation possible.
- Across thread boundaries.
- On weak executors which don't support thread-local storage.
- Across no-throw subsystem boundaries.
- Across time: save now, throw later.
- Collect, group, combine errors.
- Much simpler for a compiler to optimize.

§ 2.1. Sample Usecase

The following is how WebKit-based browsers parse URLs and use `std::expected` to denote failure:

```
template<typename CharacterType>
Expected<uint32_t, URLParser::IPv4PieceParsingError> URLParser::parseIPv4Piece(CodePointIte
rator<CharacterType>& iterator, bool& didSeeSyntaxViolation)
{
    enum class State : uint8_t {
        UnknownBase,
        Decimal,
        OctalOrHex,
        Octal,
        Hex,
    };
    State state = State::UnknownBase;
    Checked<uint32_t, RecordOverflow> value = 0;
    if (!iterator.atEnd() && *iterator == '.')
        return makeUnexpected(IPv4PieceParsingError::Failure);
    while (!iterator.atEnd()) {
        if (isTabOrNewline(*iterator)) {
            didSeeSyntaxViolation = true;
            ++iterator;
            continue;
        }
        if (*iterator == '.') {
            ASSERT(!value.hasOverflowed());
            return value.unsafeGet();
        }
        switch (state) {
        case State::UnknownBase:
            if (UNLIKELY(*iterator == '0')) {
                ++iterator;
                state = State::OctalOrHex;
                break;
            }
            state = State::Decimal;
            break;
        case State::OctalOrHex:
            didSeeSyntaxViolation = true;
            if (*iterator == 'x' || *iterator == 'X') {
```

```
                ++iterator;
                state = State::Hex;
                break;
            }
            state = State::Octal;
            break;
        case State::Decimal:
            if (!isASCIIDigit(*iterator))
                return makeUnexpected(IPv4PieceParsingError::Failure);
            value *= 10;
            value += *iterator - '0';
            if (UNLIKELY(value.hasOverflowed()))
                return makeUnexpected(IPv4PieceParsingError::Overflow);
            ++iterator;
            break;
        case State::Octal:
            ASSERT(didSeeSyntaxViolation);
            if (*iterator < '0' || *iterator > '7')
                return makeUnexpected(IPv4PieceParsingError::Failure);
            value *= 8;
            value += *iterator - '0';
            if (UNLIKELY(value.hasOverflowed()))
                return makeUnexpected(IPv4PieceParsingError::Overflow);
            ++iterator;
            break;
        case State::Hex:
            ASSERT(didSeeSyntaxViolation);
            if (!isASCIIHexDigit(*iterator))
                return makeUnexpected(IPv4PieceParsingError::Failure);
            value *= 16;
            value += toASCIIHexValue(*iterator);
            if (UNLIKELY(value.hasOverflowed()))
                return makeUnexpected(IPv4PieceParsingError::Overflow);
            ++iterator;
            break;
        }
    }
    ASSERT(!value.hasOverflowed());
    return value.unsafeGet();
}
```

These results are then accumulated in a vector , and different failure conditions are handled differently. An important fact to internalize is that the first failure encountered isn't necessarily the one which is returned, which is why exceptions aren't a good solution here: parsing must continue.

```
template<typename CharacterTypeForSyntaxViolation, typename CharacterType>
Expected<URLParser::IPv4Address, URLParser::IPv4ParsingError> URLParser::parseIPv4Host(cons
t CodePointIterator<CharacterTypeForSyntaxViolation>& iteratorForSyntaxViolationPosition, C
odePointIterator<CharacterType> iterator)
{
    Vector<Expected<uint32_t, URLParser::IPv4PieceParsingError>, 4> items;
    bool didSeeSyntaxViolation = false;
    if (!iterator.atEnd() && *iterator == '.')
        return makeUnexpected(IPv4ParsingError::NotIPv4);
    while (!iterator.atEnd()) {
        if (isTabOrNewline(*iterator)) {
            didSeeSyntaxViolation = true;
            ++iterator;
            continue;
        }
        if (items.size() >= 4)
            return makeUnexpected(IPv4ParsingError::NotIPv4);
        items.append(parseIPv4Piece(iterator, didSeeSyntaxViolation));
        if (!iterator.atEnd() && *iterator == '.') {
            ++iterator;
            if (iterator.atEnd())
                syntaxViolation(iteratorForSyntaxViolationPosition);
            else if (*iterator == '.')
                return makeUnexpected(IPv4ParsingError::NotIPv4);
        }
    }
    if (!iterator.atEnd() || !items.size() || items.size() > 4)
        return makeUnexpected(IPv4ParsingError::NotIPv4);
    for (const auto& item : items) {
        if (!item.hasValue() && item.error() == IPv4PieceParsingError::Failure)
            return makeUnexpected(IPv4ParsingError::NotIPv4);
    }
```

```
    for (const auto& item : items) {
        if (!item.hasValue() && item.error() == IPv4PieceParsingError::Overflow)
            return makeUnexpected(IPv4ParsingError::Failure);
    }
    if (items.size() > 1) {
        for (size_t i = 0; i < items.size() - 1; i++) {
            if (items[i].value() > 255)
                return makeUnexpected(IPv4ParsingError::Failure);
        }
    }
    if (items[items.size() - 1].value() >= pow256(5 - items.size()))
        return makeUnexpected(IPv4ParsingError::Failure);

    if (didSeeSyntaxViolation)
        syntaxViolation(iteratorForSyntaxViolationPosition);
    for (const auto& item : items) {
        if (item.value() > 255)
            syntaxViolation(iteratorForSyntaxViolationPosition);
    }

    if (UNLIKELY(items.size() != 4))
        syntaxViolation(iteratorForSyntaxViolationPosition);

    IPv4Address ipv4 = items.takeLast().value();
    for (size_t counter = 0; counter < items.size(); ++counter)
        ipv4 += items[counter].value() * pow256(3 - counter);
    return ipv4;
}
```

§ 2.2. Error retrieval and correction

The major advantage of `expected<T, E>` over `optional<T>` is the ability to transport an error. Programmer do the following when a function call returns an error:

1. Ignore it.
2. Delegate the responsibility of error handling to higher layer.
3. Try to resolve the error.

Because the first behavior might lead to buggy application, we ignore the usecase. The handling is dependent of the underlying error type, we consider the `exception_ptr` and the `errc` types.

§ 2.3. Impact on the standard

These changes are entirely based on library extensions and do not require any language features beyond what is available in C++20.

§ 3. Design rationale

The same rationale described in [N3672] for `optional<T>` applies to `expected<T, E>` and `expected<T, nullopt_t>` should behave almost the same as `optional<T>` (though we advise using `optional` in that case). The following sections presents the rationale in N3672 applied to `expected<T, E>`.

§ 3.1. Conceptual model of `expected<T, E>`

`expected<T, E>` models a discriminated union of types `T` and `unexpected<E>`. `expected<T, E>` is viewed as a value of type `T` or value of type `unexpected<E>`, allocated in the same storage, along with observers to determine which of the two it is.

The interface in this model requires operations such as comparison to `T`, comparison to `E`, assignment and creation from either. It is easy to determine what the value of the expected object is in this model: the type it stores (`T` or `E`) and either the value of `T` or the value of `E`.

Additionally, within the affordable limits, we propose the view that `expected<T, E>` extends the set of the values of `T` by the values of type `E`. This is reflected in initialization, assignment, ordering, and equality comparison with both `T` and `E`. In the case of `optional<T>`, `T` cannot be a `nullopt_t`. As the types `T` and `E` could be the same in `expected<T, E>`, there is need to tag the values of `E` to avoid ambiguous expressions. The `unexpected(E)` deduction guide is proposed for this purpose. However `T` cannot be `unexpected<E>` for a given `E`.

```
expected<int, string> ei = 0;
expected<int, string> ej = 1;
expected<int, string> ek = unexpected(string());

ei = 1;
ej = unexpected(E());;
ek = 0;

ei = unexpected(E());;
ej = 0;
ek = 1;
```

§ 3.2. Default `E` template paremeter type

At the Toronto meeting LEWG decided against having a default `E` template parameter type (`std::error_code` or other). This prevents us from providing an `expected` deduction guides for error construction which is a *good thing*: an error was **not** expected, a `T` was expected, it's therefore sensible to force spelling out unexpected outcomes when generating them.

§ 3.3. Initialization of `expected<T, E>`

In cases where `T` and `E` have value semantic types capable of storing `n` and `m` distinct values respectively, `expected<T, E>` can be seen as an extended `T` capable of storing `n + m` values: these `T` and `E` stores. Any valid initialization scheme must provide a way to put an expected object to any of these states. In addition, some `T`s aren't `CopyConstructible` and their expected variants still should be constructible with any set of arguments that work for `T`.

As in [N3672], the model retained is to initialize either by providing an already constructed `T` or a tagged `E`. The default constructor required `T` to be default-constructible (since `expected<T>` should behave like `T` as much as possible).

```
string s{"STR"};

expected<string, errc> es{s}; // requires Copyable<T>
expected<string, errc> et = s; // requires Copyable<T>
expected<string, errc> ev = string{"STR"}; // requires Movable<T>

expected<string, errc> ew; // expected value
expected<string, errc> ex{}; // expected value
expected<string, errc> ey = {}; // expected value
expected<string, errc> ez = expected<string,errc>{}; // expected value
```

In order to create an unexpected object, the deduction guide `unexpected` needs to be used:

```
expected<string, int> ep{unexpected(-1)}; // unexpected value, requires Movable<E>
expected<string, int> eq = unexpected(-1); // unexpected value, requires Movable<E>
```

As in [N3672], and in order to avoid calling move/copy constructor of `T`, we use a "tagged" placement constructor:

```
expected<MoveOnly, errc> eg; // expected value
expected<MoveOnly, errc> eh{}; // expected value
expected<MoveOnly, errc> ei{in_place}; // calls MoveOnly{} in place
expected<MoveOnly, errc> ej{in_place, "arg"}; // calls MoveOnly{"arg"} in place
```

To avoid calling move/copy constructor of `E`, we use a "tagged" placement constructor:

```
expected<int, string> ei{unexpect}; // unexpected value, calls string{} in place
expected<int, string> ej{unexpect, "arg"}; // unexpected value, calls string{"arg"} in plac
e
```

An alternative name for `in_place` that is coherent with `unexpect` could be `expect`. Being compatible with `optional<T>` seems more important. So this proposal doesn't propose such an `expect` tag.

The alternative and also comprehensive initialization approach, which is compatible with the default construction of `expected<T, E>` as `T()`, could have been a variadic perfect forwarding constructor that just forwards any set of arguments to the constructor of the contained object of type `T`.

§ 3.4. Never-empty guarantee

As for `boost::variant<T, unexpected<E>>`, `expected<T, E>` ensures that it is never empty. All instances `v` of type `expected<T, E>` guarantee `v` has constructed content of one of the types `T` or `E`, even if an operation on `v` has previously failed.

This implies that `expected` may be viewed precisely as a union of exactly its bounded types. This "never-empty" property insulates the user from the possibility of undefined `expected` content or an `expected valueless_by_exception` as `std::variant` and the significant additional complexity-of-use attendant with such a possibility.

In order to ensure this property the types `T` and `E` must satisfy the requirements as described in [P0110R0]. Given the nature of the parameter `E`, that is, to transport an error, it is expected to be `is_nothrow_copy_constructible<E>`, `is_nothrow_move_constructible<E>`, `is_nothrow_copy_assignable<E>` and `is_nothrow_move_assignable<E>`.

Note however that these constraints are applied only to the operations that need them.

If `is_nothrow_constructible<T, Args...>` is `false`, the `expected<T, E>::emplace(Args...)` function is not defined. In this case, it is the responsibility of the user to create a temporary and move or copy it.

§ 3.5. The default constructor

Similar data structure includes `optional<T>`, `variant<T1,...,Tn>` and `future<T>`. We can compare how they are default constructed.

- `std::optional<T>` default constructs to an optional with no value.
- `std::variant<T1,...,Tn>` default constructs to `T1` if default constructible or it is ill-formed
- `std::future<T>` default constructs to an invalid future with no shared state associated, that is, no value and no exception.
- `std::optional<T>` default constructor is equivalent to `boost::variant<nullopt_t, T>`.

This raises several questions about `expected<T, E>`:

- Should the default constructor of `expected<T, E>` behave like `variant<T, unexpected<E>>` or as `variant<unexpected<E>,T>`?
- Should the default constructor of `expected<T, nullopt_t>` behave like `optional<T>`? If yes, how should behave the default constructor of `expected<T, E>`? As if initialized with `unexpected(E())`? This would be equivalent to the initialization of `variant<unexpected<E>,T>`.
- Should `expected<T, E>` provide a default constructor at all? [N3527] presents valid arguments against this approach, e.g. `array<expected<T, E>>` would not be possible.

Requiring `E` to be default constructible seems less constraining than requiring `T` to be default constructible (e.g. consider the `Date` example in [N3527]). With the same semantics `expected<Date,E>` would be `Regular` with a meaningful not-a-date state created by default.

The authors consider the arguments in [N3527] valid for `optional<T>` and `expected<T, E>`, however the committee requested that `expected<T, E>` default constructor should behave as constructed with `T()` if `T` is default constructible.

§ 3.6. Could `Error` be `void`

`void` isn't a sensible `E` template parameter type: the `expected<T, void>` vocabularity type means "I expect a `T`, but I may have nothing for you". This is literally what `optional<T>` is for. If the error is a unit type the user can use `std::monostate` or `std::nullopt`.

§ 3.7. Conversion from `T`

An object of type `T` is implicitly convertible to an expected object of type `expected<T, E>`:

```
expected<int, errc> ei = 1; // works
```

This convenience feature is not strictly necessary because you can achieve the same effect by using tagged forwarding constructor:

```
expected<int, errc> ei{in_place, 1};
```

It has been demonstrated that this implicit conversion is dangerous [a-gotcha-with-optional].

An alternative will be to make it explicit and add a `success<T>` (similar to `unexpected<E>` explicitly convertible from `T` and implicitly convertible to `expected<T, E>`.

```
expected<int, errc> ei = success(1);
expected<int, errc> ej = unexpected(ec);
```

The authors consider that it is safer to have the explicit conversion, the implicit conversion is so friendly that we don't propose yet an explicit conversion. In addition `std::optional` has already be delivered in C++17 and it has this gotcha.

Further, having `success` makes code much more verbose than the current implicit conversion. Forcing the usage of `success` would make `expected` a much less useful vocabulary type: if success is expected then success need not be called out.

§ 3.8. Conversion from `E`

An object of type `E` is not convertible to an unexpected object of type `expected<T, E>` since `E` and `T` can be of the same type. The proposed interface uses a special tag `unexpect` and `unexpected` deduction guide to indicate an unexpected state for `expected<T, E>`. It is used for construction and assignment. This might raise a couple of objections. First, this duplication is not strictly necessary because you can achieve the same effect by using the `unexpect` tag forwarding constructor:

```
expected<string, errc> exp1 = unexpected(1);
expected<string, errc> exp2 = {unexpect, 1};
exp1 = unexpected(1);
exp2 = {unexpect, 1};
```

or simply using deduced template parameter for constructors

```
expected<string, errc> exp1 = unexpected(1);
exp1 = unexpected(1);
```

While some situations would work with the `{unexpect, ...}` syntax, using `unexpected` makes the programmer's intention as clear and less cryptic. Compare these:

```
expected<vector<int>, errc> get1() {}
    return {unexpect, 1};
}
expected<vector<int>, errc> get2() {
    return unexpected(1);
}
expected<vector<int>, errc> get3() {
    return expected<vector<int>, int>{unexpect, 1};
}
expected<vector<int>, errc> get2() {
    return unexpected(1);
}
```

The usage of `unexpected` is also a consequence of the adapted model for `expected`: a discriminated union of `T` and `unexpected<E>`.

§ 3.9. Should we support the `exp2 = {}`?

Note also that the definition of `unexpected` has an explicitly deleted default constructor. This was in order to enable the reset idiom `exp2 = {}` which would otherwise not work due to the ambiguity when deducing the right-hand side argument.

Now that `expected<T, E>` defaults to `T{}` the meaning of `exp2 = {}` is to assign `T{}`.

§ 3.10. Observers

In order to be as efficient as possible, this proposal includes observers with narrow and wide contracts. Thus, the `value()` function has a wide contract. If the expected object does not contain a value, an exception is thrown. However, when the user knows that the expected object is valid, the use of `operator*` would be more appropriated.

§ 3.11. Explicit conversion to `bool`

The rational described in [N3672] for `optional<T>` applies to `expected<T, E>`. The following example therefore combines initialization and value-checking in a boolean context.

```
if (expected<char, errc> ch = readNextChar()) {
// ...
}
```

§ 3.12. `has_value()` following P0032

`has_value()` has been added to follow [P0032R2].

§ 3.13. Accessing the contained value

Even if `expected<T, E>` has not been used in practice for enough time as `std::optional` or Boost.Optional, we consider that following the same interface as `std::optional<T>` makes the C++ standard library more homogeneous.

The rational described in [N3672] for `optional<T>` applies to `expected<T, E>`.

§ 3.14. Dereference operator

The indirection operator was chosen because, along with explicit conversion to `bool`, it is a very common pattern for accessing a value that might not be there:

```
if (p) use(*p);
```

This pattern is used for all sort of pointers (smart or raw) and `optional`; it clearly indicates the fact that the value may be missing and that we return a reference rather than a value. The indirection operator created some objections because it may incorrectly imply `expected` and `optional` are a (possibly smart) pointer, and thus provides shallow copy and comparison semantics. All library components so far use indirection operator to return an object that is not part of the pointer's/iterator's value. In contrast, `expected` as well as `optional` indirects to the part of its own state. We do not consider it a problem in the design; it is more like an unprecedented usage of indirection operator. We believe that the cost of potential confusion is overweighted by the benefit of an intuitive interface for accessing the contained value.

We do not think that providing an implicit conversion to `T` would be a good choice. First, it would require different way of checking for the empty state; and second, such implicit conversion is not perfect and still requires other means of accessing the contained value if we want to call a member function on it.

Using the indirection operator for an object that does not contain a value is undefined behavior. This behavior offers maximum runtime performance.

§ 3.15. Function value

In addition to the indirection operator, we propose the member function `value` as in [N3672] that returns a reference to the contained value if one exists or throw an exception otherwise.

```
void interact() {
    string s;
    cout << "enter number: ";
    cin >> s;
    expected<int, error> ei = str2int(s);
    try {
        process_int(ei.value());
    }
    catch(bad_expected_access<error>) {
        cout << "this was not a number.";
    }
}
```

The exception thrown is `bad_expected_access<E>` (derived from `std::exception`) which will contain the stored error.

`bad_expected_access<E>` and `bad_optional_access` could inherit both from a `bad_access` exception derived from `exception`, but this is not proposed yet.

3.16. Should `expected<T, E>::value()` throw `E` instead of `bad_expected_access<E>`?

As any type can be thrown as an exception, should `expected<T, E>` throw `E` instead of `bad_expected_access<E>`?

Some argument that standard function should throw exceptions that inherit from `std::exception`, but here the exception throw is given by the user via the type `E`, it is not the standard library that throws explicitly an exception that don't inherit from `std::exception`.

This could be convenient as the user will have directly the `E` exception. However it will be more difficult to identify that this was due to a bad expected access.

If yes, should `optional<T>` throw `nullopt_t` instead of `bad_optional_access` to be coherent?

We don't propose this.

Other have suggested to throw `system_error` if `E` is `error_code`, rethrow if `E` is `exception_ptr`, `E` if it inherits from `std::exception` and `bad_expected_access<E>` otherwise.

An alternative would be to add some customization point that state which exception is thrown but we don't propose it in this proposal. See the Appendix I.

3.17. Accessing the contained error

Usually, accessing the contained error is done once we know the expected object has no value. This is why the `error()` function has a narrow contract: it works only if `*this` does not contain a value.

```
expected<int, errc> getIntOrZero(istream_range& r) {
    auto r = getInt(); // won't throw
    if (!r && r.error() == errc::empty_stream) {
        return 0;
    }
    return r;
}
```

This behavior could not be obtained with the `value_or()` method since we want to return `0` only if the error is equal to `empty_stream`.

We could as well provide an error access function with a wide contract. We just need to see how to name each one.

3.18. Conversion to the unexpected value

The `error()` function is used to propagate errors, as for example in the next example:

```
expected<pair<int, int>, errc> getIntRange(istream_range& r) {
    auto f = getInt(r);
    if (!f) return unexpected(f.error());
    auto m = matchedString("..", r);
    if (!m) return unexpected(m.error());
    auto l = getInt(r);
    if (!l) return unexpected(l.error());
    return std::make_pair(*f, *l);
}
```

3.19. Function `value_or`

The function member `value_or()` has the same semantics than `optional` [N3672] since the type of `E` doesn't matter; hence we can consider that `E == nullopt_t` and the `optional` semantics yields.

This function is a convenience function that should be a non-member function for `optional` and `expected`, however as it is already part of the `optional` interface we propose to have it also for `expected`.

3.20. Equality operators

As for `optional` and `variant`, one of the design goals of `expected` is that objects of type `expected<T, E>` should be valid elements in STL containers and usable with STL algorithms (at least if objects of type `T` and `E` are). Equality comparison is essential for `expected<T, E>` to model concept `Regular`. C++ does not have concepts yet, but being regular is still essential for the type to be effectively used with STL.

§ 3.21. Comparison operators

Comparison operators between `expected` objects, and between mixed `expected` and `unexpected`, aren't required at this time. A future proposal could re-adopt the comparisons as defined in [P0323R2].

§ 3.22. Modifiers

§ 3.23. Resetting the value

Reseting the value of `expected<T, E>` is similar to `optional<T>` but instead of building a disengaged `optional<T>`, we build an erroneous `expected<T, E>`. Hence, the semantics and rationale is the same than in [N3672].

§ 3.24. Tag `in_place`

This proposal makes use of the "in-place" tag as defined in [C++17]. This proposal provides the same kind of "in-place" constructor that forwards (perfectly) the arguments provided to `expected`'s constructor into the constructor of `T`.

In order to trigger this constructor one has to use the tag `in_place`. We need the extra tag to disambiguate certain situations, like calling `expected`'s default constructor and requesting `T`'s default construction:

```
expected<Big, error> eb{in_place, "1"}; // calls Big{"1"} in place (no moving)
expected<Big, error> ec{in_place}; // calls Big{} in place (no moving)
expected<Big, error> ed{}; // calls Big{} (expected state)
```

§ 3.25. Tag `unexpect`

This proposal provides an "unexpect" constructor that forwards (perfectly) the arguments provided to `expected`'s constructor into the constructor of `E`. In order to trigger this constructor one has to use the tag `unexpect`.

We need the extra tag to disambiguate certain situations, notably if `T` and `E` are the same type.

```
expected<Big, error> eb{unexpect, "1"}; // calls error{"1"} in place (no moving)
expected<Big, error> ec{unexpect}; // calls error{} in place (no moving)
```

In order to make the tag uniform an additional "expect" constructor could be provided but this proposal doesn't propose it.

§ 3.26. Requirements on `T` and `E`

Class template `expected` imposes little requirements on `T` and `E`: they have to be complete object type satisfying the requirements of `Destructible`. Each operations on `expected<T, E>` have different requirements and may be disable if `T` or `E` doesn't respect these requirements. For example, `expected<T, E>`'s move constructor requires that `T` and `E` are `MoveConstructible`, `expected<T, E>`'s copy constructor requires that `T` and `E` are `CopyConstructible`, and so on. This is because `expected<T, E>` is a wrapper for `T` or `E`: it should resemble `T` or `E` as much as possible. If `T` and `E` are `EqualityComparable` then (and only then) we expect `expected<T, E>` to be `EqualityComparable`.

However in order to ensure the never empty guaranties, `expected<T, E>` requires `E` to be no throw move constructible. This is normal as the `E` stands for an error, and throwing while reporting an error is a very bad thing.

§ 3.27. Expected references

This proposal doesn't include `expected` references as `optional` C++17 doesn't include references either.

We need a future proposal.

3.28. Expected `void`

While it could seem weird to instantiate `optional` with `void`, it has more sense for `expected` as it conveys in addition, as `future<T>`, an error state. The type `expected<void, E>` means "nothing is expected, but an error could occur".

3.29. Making expected a literal type

In [N3672], they propose to make `optional` a literal type, the same reasoning can be applied to expected. Under some conditions, such that `T` and `E` are trivially destructible, and the same described for `optional`, we propose that `expected` be a literal type.

3.30. Moved from state

We follow the approach taken in `optional` [N3672]. Moving `expected<T, E>` does not modify the state of the source (valued or erroneous) of `expected` and the move semantics is up to `T` or `E`.

3.31. I/O operations

For the same reasons as `optional` [N3672] we do not add `operator<<` and `operator>>` I/O operations.

3.32. What happens when `E` is a status?

When `E` is a status, as most of the error codes are, and has more than one value that mean success, setting an `expected<T, E>` with a successful `e` value could be misleading if the user expect in this case to have also a `T`. In this case the user should use the proposed `status_value<E, T>` class. However, if there is only one value `e` that mean success, there is no such need and `expected<T, E>` compose better with the monadic interface [P0650R0].

3.33. Do we need an `expected<T, E>::error_or` function?

See [P0786R0].

Do we need to add such an `error_or` function? as member?

This function should work for all the *ValueOrError* types and so could belong to a future *ValueOrError* proposal.

Not in this proposal.

3.34. Do we need a `expected<T, E>::check_error` function?

See [P0786R0].

Do we want to add such a `check_error` function? as member?

This function should work for all the *ValueOrError* types and so could belong to a future *ValueOrError* proposal.

Not in this proposal.

3.35. Do we need a `expected<T,G>::adapt_error(function<E(G))` function?

We have the constructor `expected<T, E>(expected<T,G>)` that allows to transport explicitly the contained error as soon as it is convertible.

However sometimes we cannot change either of the error types and we could need to do this transformation. This function help to achieve this goal. The parameter is the function doing the error transformation.

This function can be defined on top of the existing interface.

```
template <class T, class E>
expected<T,G> adapt_error(expected<T, E> const& e, function<G(E)> adaptor) {
    if ( !e ) return adaptor(e.error());
    else return expected<T,G>(*e);
}
```

Do we want to add such a `adapt_error` function? as member?

This function should work for all the *ValueOrError* types and so could belong to a future *ValueOrError* proposal.

Not in this proposal.

§ 3.36. Zombie name

`unexpected` is a zombie name in C++20. It was deprecated in C++11, removed in C++17. It used to be called by the C++ runtime when a dynamic exception specification was violated.

Re-using the `unexpected` zombie name is why we have zombie names in the first place. This was discussed in the 2021-04-06 LEWG telecon, and it was pointed out that `unexpected` isn't used much in open source code besides standard library tests. The valid uses are in older code which is unlikely to update to C++23, and if it did the breakage would not be silent because the `std::unexpected()` function call wouldn't be compatible with the `std::unexpected<T>` type proposed in this paper.

§ 4. Wording

§ 4.1. Feature test macro

Add the following line to 17.3.2 [version.syn]:

```
#define __cpp_lib_execution      201903L // also in <execution>
#define __cpp_lib_expected       20yymmL // also in <expected>
#define __cpp_lib_filesystem     201703L // also in <filesystem>
```

Below, substitute the `�` character with a number or name the editor finds appropriate for the sub-section.

§ 4.2. `�`.`�` Expected objects [*expected*]

§ 4.3. `�`.`�`.1 In general [*expected.general*]

This subclause describes class template `expected` that represents expected objects. An `expected<T, E>` object holds an object of type `T` or an object of type `unexpected<E>` and manages the lifetime of the contained objects.

§ 5. `�`.`�`.2 Header `<expected>` synopsis [*expected.syn*]

```
namespace std {
    // `�`.`�`.3, class template unexpected
    template<class E> class unexpected;

    // `�`.`�`.4, class bad_expected_access
    template<class E> class bad_expected_access;

    // `�`.`�`.5, Specialization for void
    template<> class bad_expected_access<void>;

    // in-place construction of unexpected values
    struct unexpect_t{
      explicit unexpect_t() = default;
    };
    inline constexpr unexpect_t unexpect{};

    // `�`.`�`.7, class template expected
    template<class T, class E> class expected;
```

```
    // 𝔵.𝔵.8, class template expected<cv void, E>
    template<class T, class E> requires is_void_v<T> class expected<T, E>;
}
```

§ 5.1. 𝔵.𝔵.3 Unexpected objects [*expected.unexpected*]

§ 5.2. 𝔵.𝔵.3.1 General [*expected.un.general*]

This subclause describes class template `unexpected` that represents unexpected objects stored in `expected` objects.

§ 5.3. 𝔵.𝔵.3.2 Class template `unexpected` [*expected.un.object*]

```
template<class E>
class unexpected {
public:
    constexpr unexpected(const unexpected&) = default;
    constexpr unexpected(unexpected&&) = default;
    template<class... Args>
        constexpr explicit unexpected(in_place_t, Args&&...);
    template<class U, class... Args>
        constexpr explicit unexpected(in_place_t, initializer_list<U>, Args&&...);
    template<class Err = E>
        constexpr explicit unexpected(Err&&);

    constexpr unexpected& operator=(const unexpected&) = default;
    constexpr unexpected& operator=(unexpected&&) = default;

    constexpr const E& value() const& noexcept;
    constexpr E& value() & noexcept;
    constexpr const E&& value() const&& noexcept;
    constexpr E&& value() && noexcept;

    constexpr void swap(unexpected& other) noexcept(see below);

    template<class E2>
        friend constexpr bool
        operator==(const unexpected&, const unexpected<E2>&);

    friend constexpr void swap(unexpected& x, unexpected& y) noexcept(noexcept(x.swap(y)));

private:
    E val; // exposition only
};

template<class E> unexpected(E) -> unexpected<E>;
```

A program that instantiates the definition of `unexpected` for a non-object type, an array type, a specialization of `unexpected` or a cv-qualified type is ill-formed.

§ 5.3.1. 𝔵.𝔵.3.2.1 Constructors [*expected.un.ctor*]

```
template<class Err = E>
    constexpr explicit unexpected(Err&& e);
```

Constraints:

- `is_same_v<remove_cvref_t<Err>, unexpected>` is `false`; and
- `is_same_v<remove_cvref_t<Err>, in_place_t>` is `false`; and
- `is_constructible_v<E, Err>` is `true`.

Effects: Direct-non-list-initializes `val` with `std::forward<Err>(e)`.

Throws: Any exception thrown by the initialization of `val`.

```
template<class... Args>
    constexpr explicit unexpected(in_place_t, Args&&...);
```

Constraints: `is_constructible_v<E, Args...>` is `true`.

Effects: Direct-non-list-initializes `val` with `std::forward<Args>(args)...`.

Throws: Any exception thrown by the initialization of `val`.

```
template<class U, class... Args>
    constexpr explicit unexpected(in_place_t, initializer_list<U>, Args&&...);
```

Constraints: `is_constructible_v<E, initializer_list<U>&, Args...>` is `true`.

Effects: Direct-non-list-initializes `val` with `std::forward<Args>(args)...`.

Throws: Any exception thrown by the initialization of `val`.

5.3.2. ?.?.3.2.3 Observers [*expected.un.observe*]

```
constexpr const E& value() const& noexcept;
constexpr E& value() & noexcept;
```

Returns: `val`.

```
constexpr E&& value() && noexcept;
constexpr const E&& value() const&& noexcept;
```

Returns: `std::move(val)`.

5.3.3. ?.?.3.2.4 Swap [*expected.un.swap*]

```
constexpr void swap(unexpected& other) noexcept(is_nothrow_swappable_v<E>);
```

Mandates: `is_swappable_v<E>` is `true`.

Effects: Equivalent to `using std::swap; swap(val, other.val);`.

```
friend constexpr void swap(unexpected& x, unexpected& y) noexcept(noexcept(x.swap(y)));
```

Constraints: `is_swappable_v<E>` is `true`.

Effects: Equivalent to `x.swap(y)`.

5.4. ?.?.3.2.5 Equality operators [*expected.un.eq*]

```
template<class E2>
    friend constexpr bool operator==(const unexpected& x, const unexpected<E2>& y);
```

Mandates: The expression `x.value()== y.value()` is well-formed and its result is convertible to `bool`.

Returns: `x.value() == y.value()`.

5.5. ?.?.4 Class template `bad_expected_access` [*expected.bad*]

```
template<class E>
class bad_expected_access : public bad_expected_access<void> {
public:
    explicit bad_expected_access(E);
    const char* what() const noexcept override;
    E& error() & noexcept;
    const E& error() const& noexcept;
    E&& error() && noexcept;
    const E&&  error() const&& noexcept;
private:
    E val; // exposition only
};
```

The template class `bad_expected_access` defines the type of objects thrown as exceptions to report the situation where an attempt is made to access the value of `expected<T, E>` object for which `has_value()` is `false`.

```
explicit bad_expected_access(E e);
```

Effects: Initializes `val` with `std::move(e)`.

```
const E& error() const& noexcept;
E& error() & noexcept;
```

Returns: `val`.

```
E&& error() && noexcept;
const E&& error() const&& noexcept;
```

Returns: `std::move(val)`.

```
const char* what() const noexcept override;
```

Returns: An implementation-defined NTBS.

§ 5.6. 𝔼.𝔼.5 Class template specialization `bad_expected_access<void>` [*expected.bad.void*]

```
template<>
class bad_expected_access<void> : public exception {
protected:
    bad_expected_access() noexcept;
    bad_expected_access(const bad_expected_access&);
    bad_expected_access(bad_expected_access&&);
    bad_expected_access& operator=(const bad_expected_access&);
    bad_expected_access& operator=(bad_expected_access&&);
    ~bad_expected_access();

public:
    const char* what() const noexcept override;
};
```

```
const char* what() const noexcept override;
```

Returns: An implementation-defined NTBS.

§ 5.7. 𝔼.𝔼.7 Class template expected [*expected.expected*]

```
template<class T, class E>
class expected {
public:
    using value_type = T;
    using error_type = E;
    using unexpected_type = unexpected<E>;

    template<class U>
    using rebind = expected<U, error_type>;

    // 𝔼.𝔼.7.1, constructors
    constexpr expected();
    constexpr explicit(see below) expected(const expected&);
    constexpr explicit(see below) expected(expected&&) noexcept(see below);
    template<class U, class G>
        constexpr explicit(see below) expected(const expected<U, G>&);
    template<class U, class G>
        constexpr explicit(see below) expected(expected<U, G>&&);

    template<class U = T>
        constexpr explicit(see below) expected(U&& v);

    template<class G>
        constexpr expected(const unexpected<G>&);
    template<class G>
```

```
        constexpr expected(unexpected<G>&&);

    template<class... Args>
        constexpr explicit expected(in_place_t, Args&&...);
    template<class U, class... Args>
        constexpr explicit expected(in_place_t, initializer_list<U>, Args&&...);
    template<class... Args>
        constexpr explicit expected(unexpect_t, Args&&...);
    template<class U, class... Args>
        constexpr explicit expected(unexpect_t, initializer_list<U>, Args&&...);


    // ?.?.7.2, destructor
    constexpr ~expected();

    // ?.?.7.3, assignment
    constexpr expected& operator=(const expected&);
    constexpr expected& operator=(expected&&) noexcept(see below);
    template<class U = T> constexpr expected& operator=(U&&);
    template<class G>
        constexpr expected& operator=(const unexpected<G>&);
    template<class G>
        constexpr expected& operator=(unexpected<G>&&);

    // ?.?.7.4, modifiers

    template<class... Args>
        constexpr T& emplace(Args&&...) noexcept;
    template<class U, class... Args>
        constexpr T& emplace(initializer_list<U>, Args&&...) noexcept;

    // ?.?.7.5, swap
    constexpr void swap(expected&) noexcept(see below);

    // ?.?.7.6, observers
    constexpr const T* operator->() const noexcept;
    constexpr T* operator->() noexcept;
    constexpr const T& operator*() const& noexcept;
    constexpr T& operator*() & noexcept;
    constexpr const T&& operator*() const&& noexcept;
    constexpr T&& operator*() && noexcept;
    constexpr explicit operator bool() const noexcept;
    constexpr bool has_value() const noexcept;
    constexpr const T& value() const&;
    constexpr T& value() &;
    constexpr const T&& value() const&&;
    constexpr T&& value() &&;
    constexpr const E& error() const&;
    constexpr E& error() &;
    constexpr const E&& error() const&&;
    constexpr E&& error() &&;
    template<class U>
        constexpr T value_or(U&&) const&;
    template<class U>
        constexpr T value_or(U&&) &&;

    // ?.?.7.7, Expected equality operators
    template<class T2, class E2>
        friend constexpr bool operator==(const expected& x, const expected<T2, E2>& y);
    template<class T2>
        friend constexpr bool operator==(const expected&, const T2&);
    template<class E2>
        friend constexpr bool operator==(const expected&, const unexpected<E2>&);

    // ?.?.7.10, Specialized algorithms
    friend constexpr void swap(expected&, expected&) noexcept(see below);

private:
    bool has_val; // exposition only
    union
    {
        T val;  // exposition only
        E unex; // exposition only
    };
};
```

Any object of `expected<T, E>` either contains a value of type `T` or a value of type `E` within its own storage. Implementations are not permitted to use additional storage, such as dynamic memory, to

allocate the object of type `T` or the object of type `E`. These objects are allocated in a region of the `expected<T, E>` storage suitably aligned for the types `T` and `E`. Members `has_val`, `val` and `unex` are provided for exposition only. `has_val` indicates whether the `expected<T, E>` object contains an object of type `T`.

A program that instantiates the definition of template `expected<T, E>` for a reference type, a function type, or for possibly cv-qualified types `in_place_t`, `unexpect_t`, or a specialization of `unexpected` for the `T` parameter is ill-formed. A program that instantiates the definition of template `expected<T, E>` with a type for the `E` parameter that is not a valid template argument for `unexpected` is ill-formed.

When `T` is not *cv* `void`, it shall meet the requirements of `Cpp17Destructible` (Table 27).

`E` shall meet the requirements of `Cpp17Destructible` (Table 27).

§ 5.8. 𝔼.𝔼.7.1 Constructors [*expected.object.ctor*]

```
constexpr expected();
```

Constraints: `is_default_constructible_v<T>` is `true`.

Effects: Value-initializes `val`.

Postconditions: `has_value()` is `true`.

Throws: Any exception thrown by the initialization of `val`.

```
constexpr expected(const expected& rhs);
```

Effects: If `rhs.has_value()` is `true`, direct-non-list-initializes `val` with `*rhs`. Otherwise, direct-non-list-initializes `unex` with `rhs.error()`.

Postconditions: `rhs.has_value() == this->has_value()`.

Throws: Any exception thrown by the initialization of `val` or `unex`.

Remarks: This constructor is defined as deleted unless:

- `is_copy_constructible_v<T>` is `true`; and
- `is_copy_constructible_v<E>` is `true`.

This constructor is trivial if:

- `is_trivially_copy_constructible_v<T>` is `true`; and
- `is_trivially_copy_constructible_v<E>` is `true`.

```
constexpr expected(expected&& rhs) noexcept(see below);
```

Constraints:

- `is_move_constructible_v<T>` is `true`; and
- `is_move_constructible_v<E>` is `true`.

Effects: If `rhs.has_value()` is `true`, direct-non-list-initializes `val` with `std::move(*rhs)`. Otherwise, direct-non-list-initializes `unex` with `std::move(rhs.error())`.

Postconditions: `rhs.has_value()` is unchanged, `rhs.has_value() == this->has_value()`.

Throws: Any exception thrown by the initialization of `val` or `unex`.

Remarks: The exception specification is `is_nothrow_move_constructible_v<T> && is_nothrow_move_constructible_v<E>`.

Remarks: This constructor is trivial if:

- `is_trivially_move_constructible_v<T>` is `true`; and
- `is_trivially_move_constructible_v<E>` is `true`.

```
template<class U, class G>
    constexpr explicit(see below) expected(const expected<U, G>& rhs);
template<class U, class G>
    constexpr explicit(see below) expected(expected<U, G>&& rhs);
```

Let:

- `UF` be `const U&` for the first overload and `U` for the second overload.
- `GF` be `const G&` for the first overload and `G` for the second overload.

Constraints:

- `is_constructible_v<T, UF>` is `true`; and
- `is_constructible_v<E, GF>` is `true`; and
- `is_constructible_v<T, expected<U, G>&>` is `false`; and
- `is_constructible_v<T, expected<U, G>>` is `false`; and
- `is_constructible_v<T, const expected<U, G>&>` is `false`; and
- `is_constructible_v<T, const expected<U, G>>` is `false`; and
- `is_convertible_v<expected<U, G>&, T>` is `false`; and
- `is_convertible_v<expected<U, G>&&, T>` is `false`; and
- `is_convertible_v<const expected<U, G>&, T>` is `false`; and
- `is_convertible_v<const expected<U, G>&&, T>` is `false`; and
- `is_constructible_v<unexpected<E>, expected<U, G>&>` is `false`; and
- `is_constructible_v<unexpected<E>, expected<U, G>>` is `false`; and
- `is_constructible_v<unexpected<E>, const expected<U, G>&>` is `false`; and
- `is_constructible_v<unexpected<E>, const expected<U, G>>` is `false`.

Effects: If `rhs.has_value()`, direct-non-list-initializes `val` with `std::forward<UF>(*rhs)`. Otherwise, direct-non-list-initializes `unex` with `std::forward<GF>(rhs.error())`.

Postconditions: `rhs.has_value()` is unchanged, `rhs.has_value() == this->has_value()`.

Throws: Any exception thrown by the initialization of `val` or `unex`.

Remarks: The expression inside `explicit` is equivalent to `!is_convertible_v<UF, T> || !is_convertible_v<GF, E>`.

```
template<class U = T>
    constexpr explicit(!is_convertible_v<U, T>) expected(U&& v);
```

Constraints:

- `is_same_v<remove_cvref_t<U>, in_place_t>` is `false`; and
- `is_same_v<expected<T, E>, remove_cvref_t<U>>` is `false`; and
- `remove_cvref_t<U>` is not a specialization of `unexpected`; and
- `is_constructible_v<T, U>` is `true`.

Effects: Direct-non-list-initializes `val` with `std::forward<U>(v)`.

Postconditions: `has_value()` is `true`.

Throws: Any exception thrown by the initialization of `val`.

```
template<class G>
    constexpr explicit(!is_convertible_v<const G&, E>) expected(const unexpected<G>& e);
template<class G>
    constexpr explicit(!is_convertible_v<G, E>) expected(unexpected<G>&& e);
```

Let `GF` be `const G&` for the first overload and `G` for the second overload.

Constraints: `is_constructible_v<E, GF>` is `true`.

Effects: Direct-non-list-initializes `unex` with `std::forward<GF>(e.error())`.

Postconditions: `has_value()` is `false`.

Throws: Any exception thrown by the initialization of `unex`.

```
template<class... Args>
    constexpr explicit expected(in_place_t, Args&&... args);
```

Constraints:

- `is_constructible_v<T, Args...>` is `true`.

Effects: Direct-non-list-initializes `val` with `std::forward<Args>(args)...`.

Postconditions: `has_value()` is `true`.

Throws: Any exception thrown by the initialization of `val`.

```
template<class U, class... Args>
    constexpr explicit expected(in_place_t, initializer_list<U> il, Args&&... args);
```

Constraints: `is_constructible_v<T, initializer_list<U>&, Args...>` is `true`.

Effects: Direct-non-list-initializes `val` with `il, std::forward<Args>(args)...`.

Postconditions: `has_value()` is `true`.

Throws: Any exception thrown by the initialization of `val`.

```
template<class... Args>
    constexpr explicit expected(unexpect_t, Args&&... args);
```

Constraints: `is_constructible_v<E, Args...>` is `true`.

Effects: Direct-non-list-initializes `unex` with `std::forward<Args>(args)...`.

Postconditions: `has_value()` is `false`.

Throws: Any exception thrown by the initialization of `unex`.

```
template<class U, class... Args>
    constexpr explicit expected(unexpect_t, initializer_list<U> il, Args&&... args);
```

Constraints: `is_constructible_v<E, initializer_list<U>&, Args...>` is `true`.

Effects: Direct-non-list-initializes `unex` with `il, std::forward<Args>(args)...`.

Postconditions: `has_value()` is `false`.

Throws: Any exception thrown by the initialization of `unex`.

§ 5.9. 𝔵.𝔵.7.2 Destructor [*expected.object.dtor*]

```
constexpr ~expected();
```

Effects: If `has_value()` is `true`, destroys `val`, otherwise destroys `unex`.

Remarks: If `is_trivially_destructible_v<T>` is `true`, and `is_trivially_destructible_v<E>` is `true`, then this destructor is a trivial destructor.

§ 5.10. 𝔵.𝔵.7.3 Assignment [*expected.object.assign*]

This subclause makes use of the following exposition-only function:

> [Drafting note: the name *`reinit-expected`* should be kebab case in this codeblock.]

```
template<class T, class U, class... Args>
constexpr void reinit-expected(T& newval, U& oldval, Args&&... args)
{
  if constexpr (is_nothrow_constructible_v<T, Args...>) {
    destroy_at(addressof(oldval));
    construct_at(addressof(newval), std::forward<Args>(args)...);
  } else if constexpr (is_nothrow_move_constructible_v<T>) {
    T tmp(std::forward<Args>(args)...);
    destroy_at(addressof(oldval));
    construct_at(addressof(newval), std::move(tmp));
  } else {
    U tmp(std::move(oldval));
    destroy_at(addressof(oldval));
    try {
      construct_at(addressof(newval), std::forward<Args>(args)...);
```

```
    } catch (...) {
      construct_at(addressof(oldval), std::move(tmp));
      throw;
    }
  }
}
```

```
constexpr expected& operator=(const expected& rhs);
```

Effects:

- If `this->has_value() && rhs.has_value()` is `true`, equivalent to `val = *rhs`.
- Otherwise, if `this->has_value()` is `true`, equivalent to *`reinit-expected`*`(unex, val, rhs.error())`.
- Otherwise, if `rhs.has_value()` is `true`, equivalent to *`reinit-expected`*`(val, unex, *rhs)`.
- Otherwise, equivalent to `unex = rhs.error()`.

Then, if no exception was thrown, equivalent to: `has_val = rhs.has_value(); return *this;`

Remarks: This operator is defined as deleted unless:

- `is_copy_assignable_v<T>` is `true` and `is_copy_constructible_v<T>` is `true` and `is_copy_assignable_v<E>` is `true` and `is_copy_constructible_v<E>` is `true` and `is_nothrow_move_constructible_v<E> || is_nothrow_move_constructible_v<T>` is `true`.

```
constexpr expected& operator=(expected&& rhs) noexcept(see below);
```

Constraints: `is_move_constructible_v<T>` is `true` and `is_move_assignable_v<T>` is `true` and `is_move_constructible_v<E>` is `true` and `is_move_assignable_v<E>` is `true` and `is_nothrow_move_constructible_v<T> || is_nothrow_move_constructible_v<E>` is `true`.

Effects:

- If `this->has_value() && rhs.has_value()` is `true`, equivalent to `val = std::move(*rhs)`.
- Otherwise, if `this->has_value()` is `true`, equivalent to *`reinit-expected`*`(unex, val, std::move(rhs.error()))`.
- Otherwise, if `rhs.has_value()` is `true`, equivalent to *`reinit-expected`*`(val, unex, std::move(*rhs))`.
- Otherwise, equivalent to `unex = std::move(rhs.error())`.

Then, if no exception was thrown, equivalent to: `has_val = rhs.has_value(); return *this;`

Remarks: The exception specification is `is_nothrow_move_assignable_v<T> && is_nothrow_move_constructible_v<T> && is_nothrow_move_assignable_v<E> && is_nothrow_move_constructible_v<E>`.

```
template<class U = T>
    constexpr expected& operator=(U&& v);
```

Constraints:

- `is_same_v<expected, remove_cvref_t<U>>` is `false`; and
- `remove_cvref_t<U>` is not a specialization of `unexpected`; and
- `is_constructible_v<T, U>` is `true`; and
- `is_assignable_v<T&, U>` is `true`; and
- `is_nothrow_constructible_v<T, U> || is_nothrow_move_constructible_v<T> || is_nothrow_move_constructible_v<E>` is `true`.

Effects:

- If `has_value()` is `true`, equivalent to: `val = std::forward<U>(v); return *this;`
- Otherwise, equivalent to *`reinit-expected`*`(val, unex, std::forward<U>(v)); has_val = true; return *this;`

```
template<class G>
    constexpr expected& operator=(const unexpected<G>& e);
template<class G>
    constexpr expected& operator=(unexpected<G>&& e);
```

Let `GF` be `const G&` for the first overload and `G` for the second overload.

Constraints:

- `is_constructible_v<E, GF>` is `true`.
- `is_assignable_v<E&, GF>` is `true`; and
- `is_nothrow_constructible_v<E, GF> || is_nothrow_move_constructible_v<T> || is_nothrow_move_constructible_v<E>` is `true`.

Effects:

- If `has_value()` is `true`, equivalent to `reinit-expected(unex, val, std::forward<GF>(e.value())); has_val = false; return *this;`
- Otherwise, equivalent to: `unex = std::forward<GF>(e.value()); return *this;`

```
template<class... Args>
    constexpr T& emplace(Args&&... args) noexcept;
```

Constraints: `is_nothrow_constructible_v<T, Args...>` is `true`.

Effects: Equivalent to:

```
if (has_value())
  destroy_at(addressof(val));
else {
  destroy_at(addressof(unex));
  has_val = true;
}
return *construct_at(addressof(val), std::forward<Args>(args)...);
```

```
template<class U, class... Args>
    constexpr T& emplace(initializer_list<U> il, Args&&... args) noexcept;
```

Constraints: `is_nothrow_constructible_v<T, initializer_list<U>&, Args...>` is `true`.

Effects: Equivalent to:

```
if (has_value())
  destroy_at(addressof(val));
else {
  destroy_at(addressof(unex));
  has_val = true;
}
return *construct_at(addressof(val), il, std::forward<Args>(args)...);
```

§ 5.11. 𝟀.𝟀.7.4 Swap [*expected.object.swap*]

```
constexpr void swap(expected& rhs) noexcept(see below);
```

Constraints:

- `is_swappable_v<T>`; and
- `is_swappable_v<E>`; and
- `is_move_constructible_v<T> && is_move_constructible_v<E>` is `true`, and
- `is_nothrow_move_constructible_v<T> || is_nothrow_move_constructible_v<E>` is `true`.

Effects: See Table *editor-please-pick-a-number-5*

Table editor-please-pick-a-number-5 — `swap(expected&)` effects

	`has_value()`	`!has_value()`
`rhs.has_value()`	equivalent to: `using std::swap; swap(val, rhs.val);`	calls `rhs.swap(*this)`
`!rhs.has_value()`	See below [†].	equivalent to: `using std::swap; swap(unex, rhs.unex);`

[†] For the case where `rhs.value()` is `false` and `this->has_value()` is `true`, equivalent to:

```
if constexpr (is_nothrow_move_constructible_v<E>) {
  E tmp(std::move(rhs.unex));
  destroy_at(addressof(rhs.unex));
  try {
    construct_at(addressof(rhs.val), std::move(val));
    destroy_at(addressof(val));
    construct_at(addressof(unex), std::move(tmp));
  } catch(...) {
    construct_at(addressof(rhs.unex), std::move(tmp));
    throw;
  }
else {
  T tmp(std::move(val));
  destroy_at(addressof(val));
  try {
    construct_at(addressof(unex), std::move(rhs.unex));
    destroy_at(addressof(rhs.unex));
    construct_at(addressof(rhs.val), std::move(tmp));
  } catch (...) {
    construct_at(addressof(val), std::move(tmp));
    throw;
  }
}
has_val = false;
rhs.has_val = true;
```

Throws: Any exception thrown by the expressions in the *Effects*.

Remarks: The exception specification is:

```
is_nothrow_move_constructible_v<T> &&
is_nothrow_swappable_v<T> &&
is_nothrow_move_constructible_v<E> &&
is_nothrow_swappable_v<E>
```

```
friend constexpr void swap(expected& x, expected& y) noexcept(noexcept(x.swap(y)));
```

Effects: Equivalent to `x.swap(y)`.

§ 5.12. 𝟎.𝟎.7.5 Observers [*expected.object.observe*]

```
constexpr const T* operator->() const noexcept;
constexpr T* operator->() noexcept;
```

Preconditions: `has_value()` is `true`.

Returns: `addressof(val)`.

```
constexpr const T& operator*() const& noexcept;
constexpr T& operator*() & noexcept;
```

Preconditions: `has_value()` is `true`.

Returns: `val`.

```
constexpr T&& operator*() && noexcept;
constexpr const T&& operator*() const&& noexcept;
```

Preconditions: `has_value()` is `true`.

Returns: `std::move(val)`.

```
constexpr explicit operator bool() const noexcept;
constexpr bool has_value() const noexcept;
```

Returns: `has_val`.

```
constexpr const T& value() const&;
constexpr T& value() &;
```

Returns: `val`, if `has_value()` is `true` .

Throws: `bad_expected_access(error())` if `has_value()` is `false`.

```
constexpr T&& value() &&;
constexpr const T&& value() const&&;
```

Returns: `std::move(val)`, if `has_value()` is `true` .

Throws: `bad_expected_access(std::move(error()))` if `has_value()` is `false`.

```
constexpr const E& error() const& noexcept;
constexpr E& error() & noexcept;
```

Preconditions: `has_value()` is `false`.

Returns: `unex`.

```
constexpr E&& error() && noexcept;
constexpr const E&& error() const&& noexcept;
```

Preconditions: `has_value()` is `false`.

Returns: `std::move(unex)`.

```
template<class U>
    constexpr T value_or(U&& v) const&;
```

Mandates: `is_copy_constructible_v<T>` is `true` and `is_convertible<U, T>` is `true`.

Returns: `has_value() ? **this : static_cast<T>(std::forward<U>(v))`.

```
template<class U>
    constexpr T value_or(U&& v) &&;
```

Mandates: `is_move_constructible_v<T>` is `true` and `is_convertible<U, T>` is `true`.

Returns: `has_value() ? std::move(**this) : static_cast<T>(std::forward<U>(v))`.

§ 5.13. ℒ.ℒ.7.6 Expected Equality operators [*expected.object.eq*]

```
template<class T2, class E2>
    requires (!is_void_v<T2>)
    friend constexpr bool operator==(const expected& x, const expected<T2, E2>& y);
```

Mandates: The expressions `*x == *y` and `x.error()== y.error()` are well-formed and their results are convertible to `bool`.

Returns: If `x.has_value()` does not equal `y.has_value()`, `false`; otherwise if `x.has_value()` is `true`, `*x == *y`; otherwise `x.error() == y.error()`.

```
template<class T2> constexpr bool operator==(const expected& x, const T2& v);
```

Mandates: The expression `*x == v` is well-formed and its result is convertible to `bool`. [*Note:* `T1` need not be *Cpp17EqualityComparable*. - *end note*]

Returns: `x.has_value() && static_cast<bool>(*x == v)`.

```
template<class E2> constexpr bool operator==(const expected& x, const unexpected<E2>& e);
```

Mandates: The expression `x.error() == e.value()` is well-formed and its result is convertible to `bool`.

Returns: `!x.has_value() && static_cast<bool>(x.error() == e.value())`.

§ 5.14. ℒ.ℒ.8 Partial specialization of expected for void types [*expected.void*]

```
template<class T, class E> requires is_void_v<T>
class expected<T, E> {
public:
    using value_type = T;
    using error_type = E;
    using unexpected_type = unexpected<E>;

    template<class U>
    using rebind = expected<U, error_type>;

    // ??.?.8.1, constructors
    constexpr expected() noexcept;
    constexpr explicit(see below) expected(const expected&);
    constexpr explicit(see below) expected(expected&&) noexcept(see below);
    template<class U, class G>
        constexpr explicit(see below) expected(const expected<U, G>&);
    template<class U, class G>
        constexpr explicit(see below) expected(expected<U, G>&&);

    template<class G>
        constexpr expected(const unexpected<G>&);
    template<class G>
        constexpr expected(unexpected<G>&&);

    constexpr explicit expected(in_place_t) noexcept;
    template<class... Args>
        constexpr explicit expected(unexpect_t, Args&&...);
    template<class U, class... Args>
        constexpr explicit expected(unexpect_t, initializer_list<U>, Args&&...);


    // ??.?.8.2, destructor
    constexpr ~expected();

    // ??.?.8.3, assignment
    constexpr expected& operator=(const expected&);
    constexpr expected& operator=(expected&&) noexcept(see below);
    template<class G>
        constexpr expected& operator=(const unexpected<G>&);
    template<class G>
        constexpr expected& operator=(unexpected<G>&&);

    // ??.?.8.4, modifiers

    constexpr void emplace() noexcept;

    // ??.?.8.5, swap
    constexpr void swap(expected&) noexcept(see below);

    // ??.?.8.6, observers
    constexpr explicit operator bool() const noexcept;
    constexpr bool has_value() const noexcept;
    constexpr void operator*() const noexcept;
    constexpr void value() const&;
    constexpr void value() &&;
    constexpr const E& error() const&;
    constexpr E& error() &;
    constexpr const E&& error() const&&;
    constexpr E&& error() &&;

    // ??.?.8.7, Expected equality operators
    template<class T2, class E2> requires is_void_v<T2>
        friend constexpr bool operator==(const expected& x, const expected<T2, E2>& y);
    template<class E2>
        friend constexpr bool operator==(const expected&, const unexpected<E2>&);

    // ??.?.8.10, Specialized algorithms
    friend constexpr void swap(expected&, expected&) noexcept(see below);

private:
    bool has_val; // exposition only
    union
    {
        E unex;   // exposition only
    };
};
```

`E` shall meet the requirements of `Cpp17Destructible` (Table 27).

§ 5.15. ?.?.8.1 Constructors [*expected.void.ctor*]

```
constexpr expected() noexcept;
```

Postconditions: `has_value()` is `true`.

```
constexpr expected(const expected& rhs);
```

Effects: If `rhs.has_value()` is `false`, direct-non-list-initializes `unex` with `rhs.error()`.

Postconditions: `rhs.has_value() == this->has_value()`.

Throws: Any exception thrown by the initialization of `unex`.

Remarks: This constructor is defined as deleted unless `is_copy_constructible_v<E>` is `true`.

This constructor is trivial if `is_trivially_copy_constructible_v<E>` is `true`.

```
constexpr expected(expected&& rhs) noexcept(is_nothrow_move_constructible_v<E>);
```

Constraints: `is_move_constructible_v<E>` is `true`.

Effects: If `rhs.has_value()` is `false`, direct-non-list-initializes `unex` with `std::move(rhs.error())`.

Postconditions: `rhs.has_value()` is unchanged, `rhs.has_value() == this->has_value()`.

Throws: Any exception thrown by the initialization of `unex`.

Remarks: This constructor is trivial if `is_trivially_move_constructible_v<E>` is `true`.

```
template<class U, class G>
    constexpr explicit(!is_convertible_v<const G&, E>) expected(const expected<U, G>& rhs);
template<class U, class G>
    constexpr explicit(!is_convertible_v<G, E>) expected(expected<U, G>&& rhs);
```

Let `GF` be `const G&` for the first overload and `G` for the second overload.

Constraints:

- `is_void_v<U>` is `true`; and
- `is_constructible_v<E, GF>` is `true`; and
- `is_constructible_v<unexpected<E>, expected<U, G>&>` is `false`; and
- `is_constructible_v<unexpected<E>, expected<U, G>>` is `false`; and
- `is_constructible_v<unexpected<E>, const expected<U, G>&>` is `false`; and
- `is_constructible_v<unexpected<E>, const expected<U, G>>` is `false`; and

Effects: If `rhs.has_value()` is `false`, direct-non-list-initializes `unex` with `std::forward<GF>(rhs.error())`.

Postconditions: `rhs.has_value()` is unchanged, `rhs.has_value() == this->has_value()`.

Throws: Any exception thrown by the initialization of `unex`.

```
template<class G>
    constexpr explicit(!is_convertible_v<const G&, E>) expected(const unexpected<G>& e);
template<class G>
    constexpr explicit(!is_convertible_v<G, E>) expected(unexpected<G>&& e);
```

Let `GF` be `const G&` for the first overload and `G` for the second overload.

Constraints: `is_constructible_v<E, GF>` is `true`.

Effects: Direct-non-list-initializes `unex` with `std::forward<GF>(e.value())`.

Postconditions: `has_value()` is `false`.

Throws: Any exception thrown by the initialization of `unex`.

```
constexpr explicit expected(in_place_t) noexcept;
```

Postconditions: `has_value()` is `true`.

```
template<class... Args>
    constexpr explicit expected(unexpect_t, Args&&... args);
```

Constraints: `is_constructible_v<E, Args...>` is `true`.

Effects: Direct-non-list-initializes `unex` with `std::forward<Args>(args)...`.

Postconditions: `has_value()` is `false`.

Throws: Any exception thrown by the initialization of `unex`.

```
template<class U, class... Args>
    constexpr explicit expected(unexpect_t, initializer_list<U> il, Args&&... args);
```

Constraints: `is_constructible_v<E, initializer_list<U>&, Args...>` is `true`.

Effects: Direct-non-list-initializes `unex` with `il, std::forward<Args>(args)...`.

Postconditions: `has_value()` is `false`.

Throws: Any exception thrown by the initialization of `unex`.

§ 5.16. ๐.๐.8.2 Destructor [*expected.void.dtor*]

```
constexpr ~expected();
```

Effects: If `has_value()` is `false`, destroys `unex`.

Remarks: If `is_trivially_destructible_v<E>` is `true`, then this destructor is a trivial destructor.

§ 5.17. ๐.๐.8.3 Assignment [*expected.void.assign*]

```
constexpr expected& operator=(const expected& rhs);
```

Effects:

- If `this->has_value() && rhs.has_value()` is `true`, no effects.
- Otherwise, if `this->has_value()` is `true`, equivalent to: `construct_at(addressof(unex), rhs.unex); has_val = false;`
- Otherwise, if `rhs.has_value()` is `true`, destroys `unex` and sets `has_val` to `true`.
- Otherwise, equivalent to `unex = rhs.error()`.

Returns: `*this`.

Remarks: This operator is defined as deleted unless:

- `is_copy_assignable_v<E>` is `true` and `is_copy_constructible_v<E>` is `true`.

```
constexpr expected& operator=(expected&& rhs) noexcept(see below);
```

Effects:

- If `this->has_value() && rhs.has_value()` is `true`, no effects.
- Otherwise, if `this->has_value()` is `true`, equivalent to: `construct_at(addressof(unex), std::move(rhs.unex)); has_val = false;`
- Otherwise, if `rhs.has_value()` is `true`, destroys `unex` and sets `has_val` to `true`.
- Otherwise, equivalent to `unex = rhs.error()`.

Returns: `*this`.

Remarks: The exception specification is `is_nothrow_move_constructible_v<E> && is_nothrow_move_assignable_v<E>`.

This operator is defined as deleted unless:

- `is_move_constructible_v<E>` is `true` and `is_move_assignable_v<E>` is `true`.

```
template<class G>
    constexpr expected& operator=(const unexpected<G>& e);
template<class G>
    constexpr expected& operator=(unexpected<G>&& e);
```

Let `GF` be `const G&` for the first overload and `G` for the second overload.

Constraints: `is_constructible_v<E, GF>` is `true` and `is_assignable_v<E&, GF>` is `true`.

Effects:

- If `has_value()` is `true`, equivalent to: `construct_at(addressof(unex), std::forward<GF>(e.value())); has_val = false; return *this;`
- Otherwise, equivalent to: `unex = std::forward<GF>(e.value()); return *this;`

```
constexpr void emplace() noexcept;
```

Effects: If `has_value()` is `false`, destroys `unex` and sets `has_val` to `true`.

§ 5.18. 𝟎.𝟎.8.4 Swap [*expected.void.swap*]

```
constexpr void swap(expected& rhs) noexcept(see below);
```

Constraints: `is_swappable_v<E>` is `true`; and `is_move_constructible_v<E>` is `true`.

Effects: See Table *editor-please-pick-a-number-5*

Table editor-please-pick-a-number-5 — `swap(expected&)` effects

	`has_value()`	`!has_value()`
`rhs.has_value()`	no effects	calls `rhs.swap(*this)`
`!rhs.has_value()`	See below †.	equivalent to: `using std::swap; swap(unex, rhs.unex);`

† For the case where `rhs.value()` is `false` and `this->has_value()` is `true`, equivalent to:

```
construct_at(addressof(unex), std::move(rhs.unex));
destroy_at(addressof(rhs.unex));
has_val = false;
rhs.has_val = true;
```

Throws: Any exception thrown by the expressions in the *Effects*.

Remarks: The exception specification is `is_nothrow_move_constructible_v<E> && is_nothrow_swappable_v<E>`.

```
friend constexpr void swap(expected& x, expected& y) noexcept(noexcept(x.swap(y)));
```

Effects: Equivalent to `x.swap(y)`.

§ 5.19. 𝟎.𝟎.8.5 Observers [*expected.void.observe*]

```
constexpr explicit operator bool() const noexcept;
constexpr bool has_value() const noexcept;
```

Returns: `has_val`.

```
constexpr void operator*() const noexcept;
```

Preconditions: `has_value()` is `true`.

```
constexpr void value() const&;
```

Throws: `bad_expected_access(error())` if `has_value()` is `false`.

```
constexpr void value() &&;
```

Throws: `bad_expected_access(std::move(error()))` if `has_value()` is `false`.

```
constexpr const E& error() const&;
constexpr E& error() &;
```

Preconditions: `has_value()` is `false`.

Returns: `unex`.

```
constexpr E&& error() &&;
constexpr const E&& error() const&&;
```

Preconditions: `has_value()` is `false`.

Returns: `std::move(unex)`.

§ 5.20. 20.8.6 Expected Equality operators [*expected.void.eq*]

```
template<class T2, class E2>
    requires is_void_v<T2>
    friend constexpr bool operator==(const expected& x, const expected<T2, E2>& y);
```

Mandates: The expression `x.error() == y.error()` is well-formed and its result is convertible to `bool`.

Returns: If `x.has_value()` does not equal `y.has_value()`, `false`; otherwise `x.has_value() || static_cast<bool>(x.error() == y.error())`.

```
template<class E2>
    constexpr bool operator==(const expected& x, const unexpected<E2>& e);
```

Mandates: The expression `x.error() == e.value()` is well-formed and its result is convertible to `bool`.

Returns: `!x.has_value() && static_cast<bool>(x.error() == e.value())`.

§ 5.21. 16.4.5.3.2 Zombie names [*zombie.names*]

Remove `unexpected` from the list of zombie names as follows:

> In namespace `std`, the following names are reserved for previous standardization:
>
> - [...],
> - `undeclare_no_pointers`,
> - `undeclare_reachable`, and
> - ~~`unexpected`, and~~
> - `unexpected_handler`.

§ 6. Implementation & Usage Experience

There are multiple implementations of `std::expected` as specified in this paper, listed below. There are also many implementations which are similar but not the same as specified in this paper, they are not listed below.

§ 6.1. Sy Brand

By far the most popular implementation is Sy Brand's, with over 500 stars on GitHub and extensive usage.

- Code: https://github.com/TartanLlama/expected
- Non comprehensive usage list:
  - Telegram desktop client
  - Ceph distributed storage system
  - FiveM and RedM mod frameworks
  - Rspamd spam filtering system

- OTTO hardware synth
- Some NIST project
- about 10 cryptocurrency projects

* Testimonials:
  - https://twitter.com/syoyo/status/1328196033545814016

    > Ultra super cooooooooooooooooooooooooooooooooopooool!!!!! □□□□□□□□□□□□□□□□□□□□□□□□☺☺☺☺☺☺☺☺□□□□□□□ 😍😍😍😍 C++11/14/17 std::expected with functional-style extensions

  - https://twitter.com/LesleyLai6/status/1328199023786770432

    > I use @TartanLlama 's optional and expected libraries for almost all of my projects. Thay are amazing!
    > Though I made a custom fork and added a few rust Rusult like features.

  - https://twitter.com/bjorn_fahller/status/1229803982685638656

    > I used tl::expected<> and a few higher order functions to extend functionality with 1/3 code size ;-)

  - https://twitter.com/toffiloff/status/1101559543631351808

    > Also, using @TartanLlama's 'expected' library has done wonders for properly handling error cases on bare metal systems without exceptions enabled

  - https://twitter.com/chsiedentop/status/1296624103080640513

    > I can really recommend the tl::expected library which has this ☺. BTW, great library, @TartanLlama!

§ 6.2. Vicente J. Botet Escriba

The original author of `std::expected` has an implementation available:

- Code: https://github.com/viboes/std-make/blob/master/include/experimental/fundamental/v3/expected2/expected.hpp

§ 6.3. WebKit

The WebKit web browser (used in Safari) contains an implementation that's used throughout its codebase.

- Code: https://github.com/WebKit/WebKit/blob/main/Source/WTF/wtf/Expected.h

§ References

§ Informative References

[N3527]
F. Cacciola, A. Krzemieński. *A proposal to add a utility class to represent optional objects (Revision 2)*. 10 March 2013. URL: https://wg21.link/n3527

[N3672]
F. Cacciola, A. Krzemieński. *A proposal to add a utility class to represent optional objects (Revision 4)*. 19 April 2013. URL: https://wg21.link/n3672

[N3793]
F. Cacciola, A. Krzemieński. *A proposal to add a utility class to represent optional objects (Revision 5)*. 3 October 2013. URL: https://wg21.link/n3793

[N4015]
V. Escriba, P. Talbot. *A proposal to add a utility class to represent expected monad*. 26 May 2014. URL: https://wg21.link/n4015

[N4109]
V. Escriba, P. Talbot. *A proposal to add a utility class to represent expected monad - Revision 1*. 29 June 2014. URL: https://wg21.link/n4109

[P0032R2]

Vicente J. Botet Escriba. *Homogeneous interface for variant, any and optional (Revision 2)*. 13 March 2016. URL: https://wg21.link/p0032r2

[P0110R0]
Anthony Williams. *Implementing the strong guarantee for variant<> assignment*. 25 September 2015. URL: https://wg21.link/p0110r0

[P0157R0]
Lawrence Crowl. *Handling Disappointment in C++*. 7 November 2015. URL: https://wg21.link/p0157r0

[P0262r1]
Lawrence Crowl, Chris Mysen. *A Class for Status and Optional Value*. 15 October 2016. URL: https://wg21.link/p0262r1

[P0323r10]
JF Bastien, Vicente Botet. *std::expected*. 15 April 2021. URL: https://wg21.link/p0323r10

[P0323R2]
Vicente J. Botet Escriba. *A proposal to add a utility class to represent expected object (Revision 4)*. 15 June 2017. URL: https://wg21.link/p0323r2

[P0323r3]
Vicente J. Botet Escriba. *Utility class to represent expected object*. 15 October 2017. URL: https://wg21.link/p0323r3

[P0323r9]
JF Bastien, Vicente Botet. *std::expected*. 3 August 2019. URL: https://wg21.link/p0323r9

[P0343r1]
Vicente J. Botet Escriba. *Meta-programming High-Order Functions*. 15 June 2017. URL: https://wg21.link/p0343r1

[P0650R0]
Vicente J. Botet Escriba. *C++ Monadic interface*. 15 June 2017. URL: https://wg21.link/p0650r0

[P0650r2]
Vicente J. Botet Escribá. *C++ Monadic interface*. 11 February 2018. URL: https://wg21.link/p0650r2

[P0709r4]
Herb Sutter. *Zero-overhead deterministic exceptions: Throwing values*. 4 August 2019. URL: https://wg21.link/p0709r4

[P0762r0]
Niall Douglas. *Concerns about expected<T, E> from the Boost.Outcome peer review*. 15 October 2017. URL: https://wg21.link/p0762r0

[P0786R0]
Vicente J. Botet Escriba. *SuccessOrFailure, ValuedOrError and ValuedOrNone types*. 15 October 2017. URL: https://wg21.link/p0786r0

[P1028R3]
Niall Douglas. *SG14 status_code and standard error object*. 12 January 2020. URL: https://wg21.link/p1028r3

[P1051r0]
Vicente J. Botet Escribá. *std::experimental::expected LWG design issues*. 3 May 2018. URL: https://wg21.link/p1051r0

[P1095r0]
Niall Douglas. *Zero overhead deterministic failure - A unified mechanism for C and C++*. 29 August 2018. URL: https://wg21.link/p1095r0